

Zeus Learning Placement Preparation

2027 Batch Campus Drive

Vishal Chudasama · MCA, LJ University, Ahmedabad

Applying for: **Software Developer**

Company	Zeus Learning (zeuslearning.com) — ed-tech / learning technology solutions
Location	Mumbai
Roles Offered	Software Developer, Quality Engineer
Online Aptitude Test	16 September 2026
Offline Technical Test	19 September 2026
Internship Period	6 Months
Stipend	₹20,000 / month during internship
Salary (Software Developer)	7.80 LPA CTC
Bond	2 Years

Note: as per the dates in the notice, the Online Aptitude Test (16 Sep) and Offline Technical Test (19 Sep) both fall before today. If this schedule looks out of date, please confirm the current drive date with your placement coordinator — this document is still valid for revision either way.

This document contains 70 practice questions (30 Aptitude + 20 Technical + 20 Interview) built for revision before the Zeus Learning drive. The technical section focuses on DSA + Java (OOP) + DBMS, kept as language-agnostic as possible outside the Java-specific OOP questions. The aptitude section gives extra weight to Age Problems and Coding-Decoding, and adds Boats & Streams — areas flagged from earlier practice.

Contents

Section 1	Aptitude & Reasoning	30 Questions
Section 2	Technical — DSA + Java (OOP) + DBMS	20 Questions
Section 3	Extra Interview Preparation (Technical + HR)	20 Questions

Section 1 · Aptitude & Reasoning (30 Questions)

Fresher placement-level difficulty. Each question has 4 options, the correct answer marked, and a step-by-step solution where calculation is involved. Age Problems and Coding-Decoding get extra questions, and Boats & Streams is included as a new topic.

Percentage

Q1.

A student scores 45 marks out of 60 in a test. What percentage of marks did the student score?

- (A) 70%
- (B) 72%
- (C) 75%
- (D) 80%

Answer: (C) 75%

Solution: Percentage = $(45/60) \times 100 = 75\%$.

Percentage

Q2.

In a school, 40% of the students are girls. If there are 360 boys, find the total number of students.

- (A) 500
- (B) 550
- (C) 600
- (D) 650

Answer: (C) 600

Solution: Boys = 60% of total = 360.
Total = $360 \div 0.60 = 600$.

Profit & Loss

Q3.

A trader marks his goods 25% above the cost price and then allows a discount of 10%. Find his profit percentage.

- (A) 10%
- (B) 12.5%
- (C) 15%
- (D) 20%

Answer: (B) 12.5%

Solution: Let CP = 100. Marked Price = 125.
SP after 10% discount = $125 \times 0.90 = 112.5$.
Profit = $112.5 - 100 = 12.5\%$.

Profit & Loss

Q4.

By selling an article for ₹450, a shopkeeper incurs a loss of 10%. Find the cost price of the article.

- (A) ₹480
- (B) ₹500
- (C) ₹520
- (D) ₹540

Answer: (B) ₹500

Solution: SP = $0.90 \times \text{CP} \rightarrow 450 = 0.90 \times \text{CP} \rightarrow \text{CP} = ₹500$.

Ratio

Q5.

The ratio of the ages of A and B is 4:5. If the sum of their ages is 54 years, find B's age.

- (A) 24
- (B) 27
- (C) 30
- (D) 32

Answer: (C) 30

Solution: Let ages = $4x$ and $5x$. $4x+5x = 54 \rightarrow 9x = 54 \rightarrow x = 6$.
B's age = $5x = 30$.

Average

Q6.

The average marks of 25 students in a class is 60. Later it is found that one student's marks were wrongly recorded as 70 instead of the actual 20. Find the correct average.

- (A) 56
- (B) 58
- (C) 60
- (D) 62

Answer: (B) 58

Solution: Wrong total = $25 \times 60 = 1500$.
Correct total = $1500 - 70 + 20 = 1450$.
Correct average = $1450/25 = 58$.

Time & Work

Q7.

A can finish a job in 20 days and B in 30 days. They work together for 5 days, after which A leaves. How many more days will B alone take to finish the remaining work?

- (A) 15 days
- (B) 17.5 days
- (C) 18 days
- (D) 20 days

Answer: (B) 17.5 days

Solution: (A+B)'s combined 1-day work = $1/20 + 1/30 = 5/60 = 1/12$.
Work done in 5 days = $5/12$. Remaining work = $1 - 5/12 = 7/12$.
B's 1-day work = $1/30$. Days needed = $(7/12) \div (1/30) = 7/12 \times 30 = 17.5$ days.

Time & Work

Q8.

12 men can complete a piece of work in 8 days. How many men are needed to complete the same work in 6 days?

- (A) 14
- (B) 15
- (C) 16
- (D) 18

Answer: (C) 16

Solution: Men $\times$ Days = constant (total work) = $12 \times 8 = 96$.
New men needed = $96 / 6 = 16$.

Q9.

A car covers a distance of 240 km in 4 hours. Find its average speed in km/h.

- (A) 50 km/h
- (B) 55 km/h
- (C) 60 km/h
- (D) 65 km/h

Answer: (C) 60 km/h

Solution: Speed = Distance/Time = $240/4 = 60$ km/h.

Q10.

Two places A and B are 300 km apart. A train leaves A at 60 km/h heading toward B. Two hours later, another train leaves B at 90 km/h heading toward A. How long after the second train starts will the two trains meet?

- (A) 1 hour
- (B) 1.2 hours
- (C) 1.5 hours
- (D) 2 hours

Answer: (B) 1.2 hours

Solution: In the first 2 hours, train 1 covers $60 \times 2 = 120$ km.

Remaining gap when train 2 starts = $300 - 120 = 180$ km.

Combined closing speed = $60 + 90 = 150$ km/h.

Time to meet = $180/150 = 1.2$ hours.

Q11.

A boat travels 24 km downstream in 2 hours and returns the same distance upstream in 3 hours. Find the speed of the boat in still water and the speed of the stream.

- (A) Boat 9 km/h, Stream 3 km/h
- (B) Boat 10 km/h, Stream 2 km/h
- (C) Boat 11 km/h, Stream 1 km/h
- (D) Boat 12 km/h, Stream 2 km/h

Answer: (B) Boat 10 km/h, Stream 2 km/h

Solution: Downstream speed = $24/2 = 12$ km/h. Upstream speed = $24/3 = 8$ km/h.

Boat speed (still water) = (Downstream + Upstream)/2 = $(12+8)/2 = 10$ km/h.

Stream speed = (Downstream - Upstream)/2 = $(12-8)/2 = 2$ km/h.

Q12.

A man can row at 6 km/h in still water. If the speed of the stream is 2 km/h, find the time he takes to row 16 km downstream.

- (A) 1.5 hours
- (B) 2 hours
- (C) 2.5 hours
- (D) 3 hours

Answer: (B) 2 hours

Solution: Downstream speed = boat speed + stream speed = $6+2 = 8$ km/h.

Time = Distance/Speed = $16/8 = 2$ hours.

(Tip: downstream speed = boat + stream; upstream speed = boat - stream. This topic was flagged as a gap earlier — the key is remembering which sign to use.)

Simple Interest

Q13.

Find the Simple Interest on ₹12,000 at 6% p.a. for 4 years.

- (A) ₹2600
- (B) ₹2800
- (C) ₹2880
- (D) ₹3000

Answer: (C) ₹2880

Solution: $SI = (P \times R \times T)/100 = (12000 \times 6 \times 4)/100 = \text{₹}2880$.

Compound Interest

Q14.

Find the Compound Interest on ₹8,000 at 5% p.a. for 2 years, compounded annually.

- (A) ₹780
- (B) ₹800
- (C) ₹820
- (D) ₹850

Answer: (C) ₹820

Solution: Amount = $8000 \times (1.05)^2 = 8000 \times 1.1025 = 8820$.
CI = $8820 - 8000 = \text{₹}820$.

Probability

Q15.

A card is drawn at random from a well-shuffled deck of 52 playing cards. Find the probability that it is a king.

- (A) $1/13$
- (B) $1/26$
- (C) $1/4$
- (D) $4/13$

Answer: (A) $1/13$

Solution: Number of kings = 4. Total cards = 52.
 $P(\text{king}) = 4/52 = 1/13$.

Number System

Q16.

Find the unit digit of 7^{95} .

- (A) 1
- (B) 3
- (C) 7
- (D) 9

Answer: (B) 3

Solution: Unit digits of powers of 7 cycle every 4 terms: 7, 9, 3, 1, 7, 9, 3, 1...
 $95 \div 4 = 23$ remainder 3 $\rightarrow$ the unit digit matches the **3rd** term in the cycle, which is **3**.

HCF & LCM

Q17.

The HCF and LCM of two numbers are 6 and 360 respectively. If one of the numbers is 45, find the other number.

- (A) 40
- (B) 45
- (C) 48
- (D) 50

Answer: (C) 48

Solution: Product of two numbers = HCF $\times$ LCM = $6 \times 360 = 2160$.
Other number = $2160 / 45 = 48$.

Q18.

A father is 4 times as old as his son. After 20 years, the father will be twice as old as his son. Find the son's present age.

- (A) 8
- (B) 10
- (C) 12
- (D) 15

Answer: (B) 10

Solution: Let son = x , father = $4x$.

$$4x+20 = 2(x+20) \rightarrow 4x+20 = 2x+40 \rightarrow 2x = 20 \rightarrow x = 10.$$

(Father's present age = 40.)

Q19.

The sum of the ages of a mother and daughter is 46 years. Five years ago, the mother's age was 5 times the daughter's age. Find the daughter's present age.

- (A) 9
- (B) 11
- (C) 13
- (D) 15

Answer: (B) 11

Solution: Let daughter = d , mother = m . $m+d = 46$.

Five years ago: $m-5 = 5(d-5) \rightarrow m = 5d-20$.

Substitute: $5d-20+d = 46 \rightarrow 6d = 66 \rightarrow d = 11$.

(Mother's present age = 35. Check: 5 years ago, mother=30, daughter=6, $30 = 5 \times 6$ ✓)

Q20.

The present ages of A and B are in the ratio 5:7. Eight years from now, the ratio of their ages will be 3:4. Find A's present age.

- (A) 32
- (B) 35
- (C) 40
- (D) 45

Answer: (C) 40

Solution: Let ages = $5x$ and $7x$.

$$(5x+8)/(7x+8) = 3/4 \rightarrow 20x+32 = 21x+24 \rightarrow x = 8.$$

A's present age = $5x = 40$.

Q21.

Rohan is twice as old as his brother Sohan. Four years ago, Rohan was three times as old as Sohan. Find their present ages.

- (A) Rohan 14, Sohan 7
- (B) Rohan 16, Sohan 8
- (C) Rohan 18, Sohan 9
- (D) Rohan 20, Sohan 10

Answer: (B) Rohan 16, Sohan 8

Solution: Let Sohan = x , Rohan = $2x$.

$$2x-4 = 3(x-4) \rightarrow 2x-4 = 3x-12 \rightarrow x = 8.$$

Sohan = 8, Rohan = $2x = 16$.

Number Series

Q22.

Find the next number in the series: 5, 11, 23, 47, 95, ?

- (A) 189
- (B) 190
- (C) 191
- (D) 192

Answer: (C) 191

Solution: Each term follows the rule: (previous term $\times$ 2) + 1.
 $5 \times 2 + 1 = 11$, $11 \times 2 + 1 = 23$, $23 \times 2 + 1 = 47$, $47 \times 2 + 1 = 95$, $95 \times 2 + 1 = \mathbf{191}$.

Letter Series

Q23.

Find the missing term: AZ, BY, CX, DW, ?

- (A) EV
- (B) EU
- (C) FV
- (D) EW

Answer: (A) EV

Solution: The first letter moves forward (A,B,C,D,E...) while the second letter moves backward from Z (Z,Y,X,W,V...).
Next pair = **EV**.

Coding-Decoding

Q24.

In a certain code, 'ROSE' is written as 'URVH'. How is 'TIGER' written in that code?

- (A) WLJHU
- (B) WLJHV
- (C) WKJHU
- (D) WLIHU

Answer: (A) WLJHU

Solution: Each letter is shifted forward by 3: R $\rightarrow$ U, O $\rightarrow$ R, S $\rightarrow$ V, E $\rightarrow$ H.
Applying the same shift to TIGER: T $\rightarrow$ W, I $\rightarrow$ L, G $\rightarrow$ J, E $\rightarrow$ H, R $\rightarrow$ U $\rightarrow$ **WLJHU**.

Coding-Decoding

Q25.

If 'PENCIL' is coded as 'QDOBJK', what is the coding pattern, and how would 'BOOK' be coded using the same pattern?

- (A) CNPJ
- (B) CNPK
- (C) BNPJ
- (D) CNOJ

Answer: (A) CNPJ

Solution: The letters alternate +1, -1, +1, -1...: P(+1) $\rightarrow$ Q, E(-1) $\rightarrow$ D, N(+1) $\rightarrow$ O, C(-1) $\rightarrow$ B, I(+1) $\rightarrow$ J, L(-1) $\rightarrow$ K.
Applying to BOOK: B(+1) $\rightarrow$ C, O(-1) $\rightarrow$ N, O(+1) $\rightarrow$ P, K(-1) $\rightarrow$ J $\rightarrow$ **CNPJ**.

Coding-Decoding

Q26.

In a certain code, letters are represented by their position in the alphabet (A=1, B=2, ... Z=26). If 'CAT' is written as '3-1-20', what does '4-15-7' represent?

- (A) DOG
- (B) DOT
- (C) FOG
- (D) LOG

Answer: (A) DOG

Solution: 4=D, 15=O, 7=G $\rightarrow$ **DOG**.

Q27.

In a certain code, each letter of a word is converted to its alphabet position and the positions are summed. If 'CAT' is coded as 24 (C=3, A=1, T=20; $3+1+20=24$), what is the code for 'DOG' using the same logic?

- (A) 24
- (B) 25
- (C) 26
- (D) 28

Answer: (C) 26

Solution: D=4, O=15, G=7. Sum = $4+15+7 = 26$.

Q28.

Statements: Some doctors are engineers. All engineers are graduates. **Conclusion:** Some doctors are graduates. Is this conclusion valid?

- (A) Valid
- (B) Invalid
- (C) Cannot be determined
- (D) Partially valid

Answer: (A) Valid

Solution: The doctors who are engineers are, by the second statement, also graduates. So it follows that some doctors (at least those who are engineers) are graduates — the conclusion is **valid**.

Q29.

Find the odd one out: Triangle, Square, Circle, Speed.

- (A) Triangle
- (B) Square
- (C) Circle
- (D) Speed

Answer: (D) Speed

Solution: Triangle, Square and Circle are all geometric shapes; Speed is not a shape — it is the odd one out.

Q30.

Choose the correctly spelled word.

- (A) Occassion
- (B) Occasion
- (C) Ocassion
- (D) Occasionn

Answer: (B) Occasion

Solution: The correct spelling has a double 'c' and a single 's': **Occasion**.

Section 2 · Technical — DSA + Java (OOP) + DBMS (20 Questions)

Since Zeus Learning's notice doesn't name a specific tech stack for the Software Developer role, this section focuses on core fundamentals: Data Structures & Algorithms (language-agnostic, with Java where code is shown), Java-based OOP, and DBMS/SQL — the areas most general "offline technical test" formats tend to cover for freshers.

DSA

D1.

What is time complexity, and why does it matter? Explain with common Big-O examples.

Answer: Time complexity describes how the running time of an algorithm grows as the input size (n) grows. It matters because it lets you compare algorithms independently of hardware or specific input — an algorithm that's fast for small inputs might become unusable for large ones.

Common complexities, from fastest to slowest growth:

- **$O(1)$** — constant time (e.g., accessing an array element by index).
- **$O(\log n)$** — logarithmic (e.g., binary search).
- **$O(n)$** — linear (e.g., a single loop through an array).
- **$O(n \log n)$** — e.g., efficient sorting algorithms like merge sort.
- **$O(n^2)$** — quadratic (e.g., nested loops, bubble sort).

DSA

D2.

What is the difference between an Array and a Linked List?

Answer:

- **Array** — stores elements in contiguous memory; access by index is $O(1)$, but inserting/deleting in the middle is $O(n)$ since elements must shift, and size is typically fixed.
 - **Linked List** — stores elements as nodes, each pointing to the next; insertion/deletion is $O(1)$ once you have a reference to the position, but access by index is $O(n)$ since you must traverse from the head, and it uses extra memory for pointers.
- Use arrays when you need fast random access; use linked lists when you need frequent insertions/deletions.

DSA

D3.

What is the difference between a Stack and a Queue? Give one real-world use case for each.

Answer:

- **Stack** — LIFO (Last In, First Out). Elements are added and removed from the same end (the "top"). Use case: the "undo" feature in an editor, or function call stacks during recursion.
- **Queue** — FIFO (First In, First Out). Elements are added at the "rear" and removed from the "front". Use case: task scheduling, or handling requests in the order they arrive (e.g., a print queue).

DSA

D4.

Explain Binary Search. What is its time complexity, and why is it faster than linear search?

Answer: Binary Search finds a target value in a **sorted** array by repeatedly comparing the target to the middle element and eliminating half the remaining search space each time.

```
int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // not found
}
```

Its time complexity is **$O(\log n)$** , much faster than linear search's $O(n)$, because each comparison halves the remaining elements to check — but it requires the array to be sorted first.

D5.

What is recursion? Explain with a factorial example, including the base case.

Answer: Recursion is when a function calls itself to solve a smaller instance of the same problem, until it reaches a **base case** that stops further calls.

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;    // base case
    return n * factorial(n - 1);       // recursive case
}
// factorial(5) = 5*4*3*2*1 = 120
```

Without a correctly defined base case, a recursive function would call itself forever and cause a stack overflow.

D6.

Write a function to reverse an array in place (without using a built-in reverse method).

Answer (Java):

```
void reverseArray(int[] arr) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}
// Input:  [1, 2, 3, 4, 5]
// Output: [5, 4, 3, 2, 1]
```

Explanation: Use two pointers, one at each end, and swap elements while moving them toward the center — this reverses the array in $O(n)$ time with $O(1)$ extra space.

D7.

What is a Binary Search Tree (BST)? What is the time complexity of search and insertion?

Answer: A BST is a tree where, for every node, all values in its **left subtree** are smaller and all values in its **right subtree** are larger than the node itself. This ordering allows searching by repeatedly moving left or right, similar to binary search.

- **Average case** (balanced tree): search/insert is **$O(\log n)$** .
 - **Worst case** (skewed tree, e.g., inserting sorted data one by one): it degrades to a linked list, giving **$O(n)$** .
- Self-balancing trees (like AVL or Red-Black trees) exist specifically to guarantee $O(\log n)$ even in the worst case.

D8.

Explain the difference between Bubble Sort and Quick Sort, including their time complexities.

Answer:

- **Bubble Sort** — repeatedly compares adjacent elements and swaps them if out of order, "bubbling" the largest element to the end each pass. Simple but inefficient: **$O(n^2)$** in average and worst case.
 - **Quick Sort** — picks a "pivot" element, partitions the array so smaller elements go left and larger go right of the pivot, then recursively sorts each side. Average case is **$O(n \log n)$** , though worst case (e.g., a poorly chosen pivot on already-sorted data) is $O(n^2)$.
- In practice, Quick Sort is much faster on large datasets, which is why it (or variants like Timsort) is commonly used in real sorting libraries.

J1.

Explain the four pillars of OOP with a short Java example for each.

Answer:

- **Encapsulation** — keeping fields private and exposing controlled access via getters/setters.

```
class Account {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amt) { if (amt > 0) balance += amt; }
}
```

- **Abstraction** — hiding implementation details behind an abstract class or interface.

```
abstract class Shape {
    abstract double area(); // implementation hidden from caller
}
```

- **Inheritance** — a subclass reuses/extends a parent class.

```
class Employee { String name; }
class Manager extends Employee { int teamSize; }
```

- **Polymorphism** — the same method behaves differently depending on the object (method overriding shown below).

```
class Animal { void sound() { System.out.println("Some sound"); } }
class Dog extends Animal { void sound() { System.out.println("Bark"); } }
```

J2.

What is the difference between method overloading and method overriding in Java?

Answer:

- **Overloading** — multiple methods in the **same class** with the same name but different parameter lists (number/type of arguments). Resolved at **compile time**.

```
void add(int a, int b) { }
void add(double a, double b) { }
```

- **Overriding** — a subclass provides its own implementation of a method already defined in its parent class, with the **same signature**. Resolved at **runtime** (dynamic/ run-time polymorphism), and requires the `@Override` annotation as good practice.

```
class Animal { void sound() { System.out.println("Some sound"); } }
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }
}
```

J3.

What is the difference between an abstract class and an interface in Java?

Answer:

- An **abstract class** can have both abstract methods and fully implemented ones, can hold instance fields/state, and a class can extend only **one** abstract class.
 - An **interface** traditionally only declared method signatures (Java 8+ also allows `default` and `static` methods with implementation), cannot hold instance state (only `public static final` constants), and a class can implement **multiple** interfaces.
- Rule of thumb: use an abstract class when subclasses share common code/state; use an interface to define a capability/contract that unrelated classes can all implement.

J4.**What is a constructor in Java? Explain the difference between a default and a parameterized constructor.**

Answer: A constructor is a special block of code, matching the class name, that runs automatically when an object is created — used to initialize the object's state.

- **Default constructor** — takes no arguments; if you don't write any constructor, Java provides one automatically.
- **Parameterized constructor** — takes arguments so you can initialize fields with specific values at creation time.

```
class Student {
    String name;
    Student() { name = "Unknown"; }           // default
    Student(String name) { this.name = name; } // parameterized
}
```

J5.**What is the difference between the "this" and "super" keywords in Java?**

Answer:

- **this** refers to the **current object** — commonly used to distinguish an instance field from a parameter of the same name, or to call another constructor in the same class.
- **super** refers to the **parent class** — used to call the parent's constructor, or to access a parent method/field that's been overridden or hidden.

```
class Animal {
    Animal() { System.out.println("Animal created"); }
    void eat() { System.out.println("Animal eats"); }
}
class Dog extends Animal {
    Dog() {
        super();           // calls Animal's constructor
        System.out.println("Dog created");
    }
    void eat() {
        super.eat();       // calls Animal's eat()
        System.out.println("Dog eats too");
    }
}
```

J6.**What is exception handling in Java? Explain try-catch-finally with an example.**

Answer: Exception handling lets a program deal with runtime errors gracefully instead of crashing. Code that might throw an error is placed in a **try** block; a matching **catch** block handles the specific exception type; a **finally** block always runs afterward (e.g., to close a file/connection), whether or not an exception occurred.

```
try {
    int result = 10 / 0;           // throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
    System.out.println("This always runs.");
}
```

B1.**What is the difference between DBMS and RDBMS?**

Answer: A **DBMS** (Database Management System) is software for storing and managing data, but it doesn't necessarily enforce relationships or a tabular structure (e.g., a simple file-based system). An **RDBMS** (Relational DBMS) stores data in structured **tables** with rows and columns, enforces relationships between tables via keys, and supports SQL and constraints like primary/foreign keys, uniqueness, and normalization rules. MySQL, PostgreSQL, and Oracle are all RDBMS.

B2.**What is the difference between a Primary Key and a Foreign Key?****Answer:**

- A **Primary Key** uniquely identifies each row in its own table — it cannot be NULL and must be unique.
- A **Foreign Key** is a column in one table that references the Primary Key of another table, used to establish and enforce a relationship between the two tables (e.g., an `orders.customer_id` column referencing `customers.id`).

B3.**What is normalization? Briefly explain 1NF, 2NF, and 3NF with an example.**

Answer: Normalization organizes data to reduce redundancy and avoid update anomalies, usually by splitting data across related tables.

- **1NF** (First Normal Form) — each column holds atomic (indivisible) values, no repeating groups (e.g., don't store "Math, Science" in one cell — split into separate rows).
- **2NF** — must be in 1NF, and every non-key column must depend on the **whole** primary key (relevant when the key is composite).
- **3NF** — must be in 2NF, and no non-key column should depend on another non-key column (no transitive dependency) — e.g., a student's "city" shouldn't be stored based on their "pincode" in the same table; pincode-to-city should live in a separate lookup table.

B4.**What is indexing in a database, and how does it improve query performance?**

Answer: An index is a separate data structure (commonly a B-Tree) built on one or more columns that lets the database find rows without scanning the entire table — similar to an index at the back of a book. Without an index, a query like `WHERE email = 'x@y.com'` forces a full table scan ($O(n)$); with an index on `email`, lookups become much faster (roughly $O(\log n)$). The trade-off is that indexes take extra storage and slightly slow down writes (INSERT/UPDATE/DELETE), since the index must also be updated.

B5.**What is a transaction in DBMS? Explain the ACID properties.**

Answer: A transaction is a sequence of one or more database operations that are executed as a single logical unit of work — either all of them succeed, or none do. ACID properties guarantee reliable transactions:

- **Atomicity** — all operations in the transaction complete, or none do (all-or-nothing).
- **Consistency** — a transaction takes the database from one valid state to another, respecting all rules/constraints.
- **Isolation** — concurrently running transactions don't interfere with each other's intermediate state.
- **Durability** — once committed, changes persist even if the system crashes right after.

B6.**Write an SQL query to find duplicate email entries in a "users" table.****Answer:**

```
SELECT email, COUNT(*) AS occurrences
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation: `GROUP BY email` groups all rows sharing the same email address, and `HAVING COUNT(*) > 1` filters that grouped result down to only the emails that appear more than once.

Section 3 · Extra Interview Preparation (20 Questions)

A mix of technical follow-ups and HR/behavioural questions specific to this drive — including relocation to Mumbai, the 6-month internship stipend, and the 2-year bond. Sample answers are written in first person as a starting point; personalize them further before the interview.

HR

I1.

Tell me about yourself.

Sample answer: "I'm Vishal, currently pursuing my MCA at LJ University in Ahmedabad, after completing my BCA at JJKCC. I'm a hands-on learner — alongside coursework, I maintain personal repositories where I practice DSA, Java, and web development, and I've built projects like CarZen, a Flutter and FastAPI/MySQL-based car marketplace app, and SecureVaultX, a secure file-storage project built with a proper three-tier architecture. I'm now looking to start my career as a Software Developer, and Zeus Learning's work in ed-tech — building learning platforms and tools — is something I find genuinely interesting to contribute to."

HR

I2.

Why do you want to work as a Software Developer?

Sample answer: "I enjoy the process of turning a problem into a working solution — breaking it down, structuring the data and logic cleanly, and seeing it come together as something people actually use. I've built that skill through my coursework and personal projects, practicing data structures, algorithms, and full-stack development, and I want to keep growing that as a full-time developer."

HR

I3.

Why do you want to join Zeus Learning specifically?

Sample answer: "Zeus Learning has spent 25 years focused specifically on ed-tech, building things like Learning Management Systems, assessment tools, and virtual classrooms for well-known publishers and institutions. As someone currently in an academic environment myself, I find it genuinely appealing to work on products that directly shape how people learn, rather than a generic product with no particular domain focus. It also gives me the chance to work across a range of technologies, which is great for a fresher who wants to keep learning."

HR

I4.

What do you know about Zeus Learning's products and clients?

Sample answer: "Zeus Learning builds learning technology solutions mainly for the North American and European education markets — things like Learning Management Systems, assessment and TEI (technology-enhanced item) systems, software skills simulations, virtual classrooms, learning portals, mobile apps, and interactive content like learning games. They've worked with major names like McGraw-Hill, Cengage Learning, EBSCO, National Geographic, Discovery Education, and Harvard Business Publishing, which shows the scale and credibility of the clients they support."

HR

I5.

What are your key strengths?

Sample answer: "I'm a self-driven learner — I don't just study concepts for exams, I build small projects to actually apply them, which is why I have several personal repositories and pinned GitHub projects. I'm also methodical when debugging or solving a new problem — I'd rather work through it step by step than guess."

HR

I6.

What do you consider your biggest weakness?

Sample answer: "Since I like building things thoroughly on my own, I sometimes spend more time polishing a personal project than strictly necessary. I've been addressing this by setting clearer time-boxes for myself before starting a task, so I can ship something working first and refine it afterward."

HR

I7.

Why should we hire you?

Sample answer: "I bring hands-on exposure to both frontend and backend development, along with a solid grip on data structures and OOP fundamentals, built through my own projects rather than just coursework. I pick up new codebases quickly, ask the right questions when I'm unsure, and genuinely enjoy the process of building software — which means I'll keep improving fast on the job."

HR · Drive-specific

I8.

Are you comfortable relocating to Mumbai for this role?

Sample answer: "Yes, I'm comfortable relocating to Mumbai for this opportunity. I understand it's a different city from where I'm currently based in Ahmedabad, but I see this as a good opportunity to start my career, and I'm prepared to make that move."

(Be honest here — if relocating is genuinely difficult for you, it's better to raise it early and ask about remote/hybrid options than to commit and struggle later.)

HR · Drive-specific

I9.

This role has a 6-month internship period with a stipend of ₹20,000/month before the full CTC of 7.80 LPA kicks in. Are you comfortable with that?

Sample answer: "Yes, I understand internship periods are standard for freshers and are meant as a mutual evaluation window. I see it as a chance to learn Zeus Learning's codebase, tools, and processes properly before taking on full responsibilities, and I'm comfortable with the stipend during that period."

HR · Drive-specific

I10.

Are you okay with the 2-year service bond for this role?

Sample answer: "Yes, I understand the bond reflects the company's investment in training a fresher, and I'm committed to building my career here for that period. I'd just want to fully understand the exact terms — what the bond document specifies and what happens in edge cases — before signing, which I think is a reasonable thing to clarify before accepting any offer."

HR

I11.

Where do you see yourself in the next 3-5 years?

Sample answer: "I want to grow from a fresher developer into someone who can independently own a feature or module end-to-end, with a solid understanding of both the technical side and the product/domain it serves — in this case, ed-tech. Longer term, I'd like to also mentor newer freshers joining the team."

Project

I12.

Tell me about a project you have built.

Sample answer: "One project I worked on is CarZen, a car buying/selling marketplace app with an existing FastAPI and MySQL backend, where my part was building and polishing the Flutter frontend — screens like Browse Cars, Car Details, Login/Register, Orders, and an Admin panel — making sure it was fully driven by real backend data and responsive across devices. It's also part of my MCA coursework as a group project. I've also built SecureVaultX, a personal project for secure file storage using a clean three-tier architecture (Spring Boot backend, Flutter frontend), which helped me practice structuring a backend properly and handling authentication and encryption concepts. Alongside these, I maintain separate practice repositories for DSA problems and Java, which is where a lot of my problem-solving fundamentals come from."

Project

I13.

What was the most challenging part of that project, and how did you solve it?

Sample answer: "With CarZen, the biggest challenge was making sure the frontend only ever reflected what the backend actually supported — I had to treat the existing backend code and API documentation as the source of truth instead of guessing endpoint names or response fields, and work incrementally rather than rewriting large parts of the app at once. It taught me to read existing code carefully before changing anything, which is a habit I now carry into any codebase I join."

Problem-Solving

I14.

How do you approach solving a new coding problem you haven't seen before?

Sample answer: "First, I make sure I understand the problem fully — I restate it in my own words and check the constraints and edge cases (empty input, very large input, duplicates, etc.). Then I think through a brute-force approach first, even if it's not optimal, just to make sure I have something correct. Only after that do I look for ways to optimize it — often by identifying if it maps to a known pattern like two-pointers, sliding window, or a particular data structure. I test with a couple of example inputs by hand before assuming the logic is correct."

OOP

I15.

Can you give a real example of inheritance or polymorphism from one of your own projects?

Sample answer: "In SecureVaultX, different types of stored items (like a vault file versus a direct encrypt-and-download file) share common behavior — like validation and encryption steps — but differ in how they're persisted. Structuring that as a base type with shared logic and more specific subtypes/handlers for each mode is a practical example of using inheritance and polymorphism to avoid duplicating the shared logic while still allowing each mode to behave differently where it needs to."

DBMS · Applied

I16.

How would you design a simple database schema for an online quiz/course platform?

Sample answer: "At a minimum, I'd start with tables like `users` (id, name, email, role), `courses` (id, title, description), `quizzes` (id, course_id as a foreign key), `questions` (id, quiz_id, question_text), `options` (id, question_id, option_text, is_correct), and `attempts` (id, user_id, quiz_id, score, attempted_at). Foreign keys tie quizzes to courses, questions to quizzes, and attempts to both users and quizzes, so I can query things like 'a user's score history' or 'all questions in a quiz' efficiently, while normalizing the data so a question's text isn't duplicated across every user's attempt."

Coding

I17.

Write a function to count the number of vowels in a given string.

Answer (Java):

```
int countVowels(String str) {
    int count = 0;
    String vowels = "aeiouAEIOU";
    for (char c : str.toCharArray()) {
        if (vowels.indexOf(c) != -1) {
            count++;
        }
    }
    return count;
}
// countVowels("Zeus Learning") -> 5
```

Debugging

I18.

How do you approach debugging a program that isn't behaving as expected?

Sample answer: "I start by reproducing the issue reliably, then narrow down where it's happening — using print statements, logs, or a debugger to check the actual values at each step versus what I expect. I try to isolate the smallest piece of code that reproduces the bug rather than staring at the whole program, and I check recent changes first if it was working before. Once I understand the root cause, I fix it and then re-test the surrounding cases too, in case the same mistake shows up elsewhere."

Git / Collaboration

I19.

Have you worked with Git/GitHub for version control or team collaboration?

Sample answer: "Yes, I maintain a public GitHub profile with several personal and coursework repositories, and I've used Git for day-to-day version control — committing incrementally, working with branches, and resolving merge conflicts when needed. For a group project like CarZen, I've coordinated changes with a teammate so we don't overwrite each other's backend or database work, which taught me to communicate clearly about who's touching what before making changes."

HR

I20.

How do you handle tight deadlines or work pressure?

Sample answer: "I break the task into smaller steps as early as possible and tackle the most uncertain or riskiest part first, rather than leaving it for last. I also believe in flagging blockers early rather than staying stuck silently — I'd rather ask a quick clarifying question than submit something incorrect at the deadline."

Bonus Tip

Prepare 2-3 genuine questions for the interviewer — e.g., "What does the onboarding process look like for new Software Developers during the internship period?" or "What kind of tech stack does the team I'd join typically work with?" Given Zeus Learning's ed-tech focus, showing genuine curiosity about their products (LMS, assessment tools, virtual classrooms) can also help you stand out.

Prepared for Vishal — Zeus Learning 2027 Batch Placement Drive. All aptitude calculations have been independently verified. The Online Aptitude Test (16 Sep) and Offline Technical Test (19 Sep) dates in the original notice appear to have already passed as of when this was prepared — please confirm the current schedule with your placement coordinator.